# Random Forest

15 October 2025 21:07

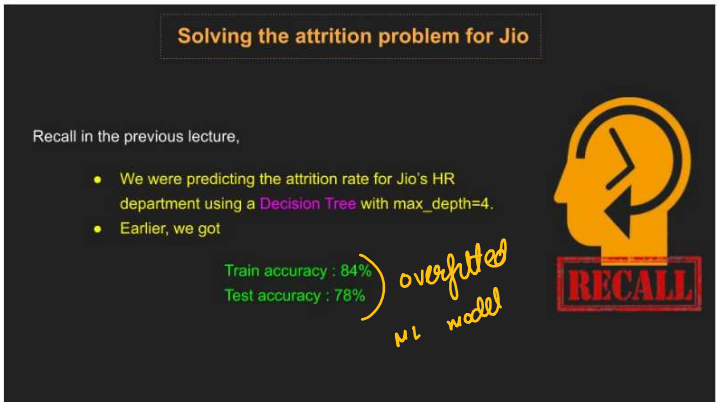
Agenda → Individual D-Tree

→ (Ensembles methods) → Random Forest
→ Boosting

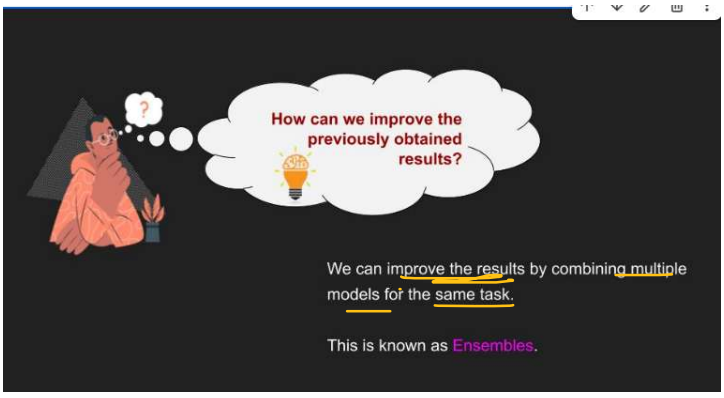
→ Bagging & introduce RF

→ OOB error / score

→ code implemen^n




Ques → Can we combine multiple D-Trees to create a stronger model




(Ensemble) | English)
(group)/collection

→ Analytics PoV
Ensemble
↓
many ML models for same task

many ML models for same task
+
Combining them smartly

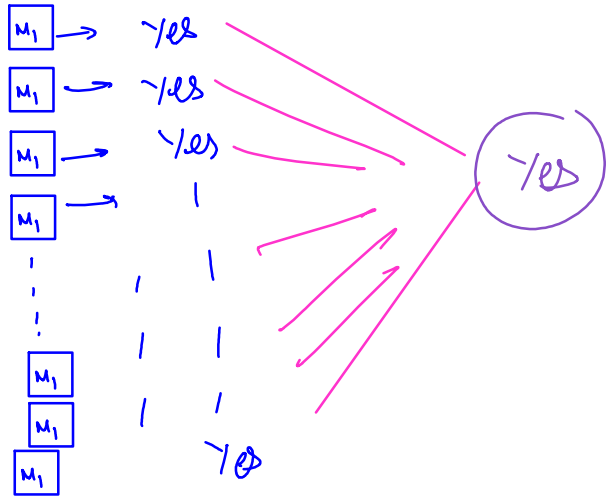
Ques D =

| $x_1 - - x_d$ | Y |
|---|---|
| | |

$D = \{(x_i, y_i) \forall\ i = 1 \text{ to } N\}$

If you train 100 d. trees → Normal D-Tree

100 Trained models







# Concept → since data is same, (X) and (Y) is same for 100 models, my output will be same as individual model

Ques * we need either diff data to train or something else

D

What is Bagging = (Bootstrap + Aggregation)

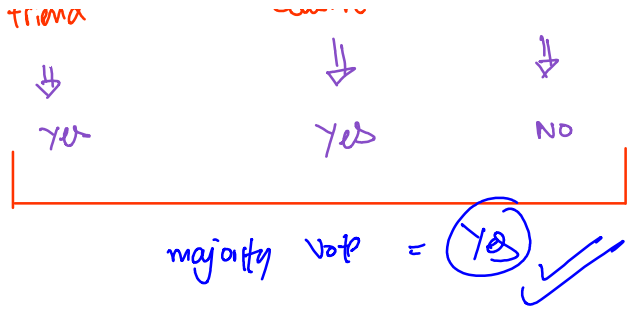
Task

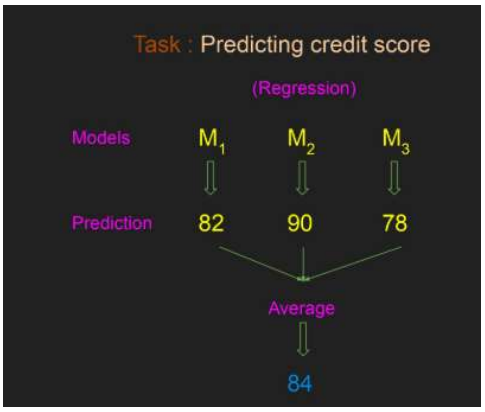
Suppose, you want to buy an iphone. You ask these people for their opinion:

M1 Friend

M2 Cousin

(M3) Mother




Q If models are same, what will be the output ?

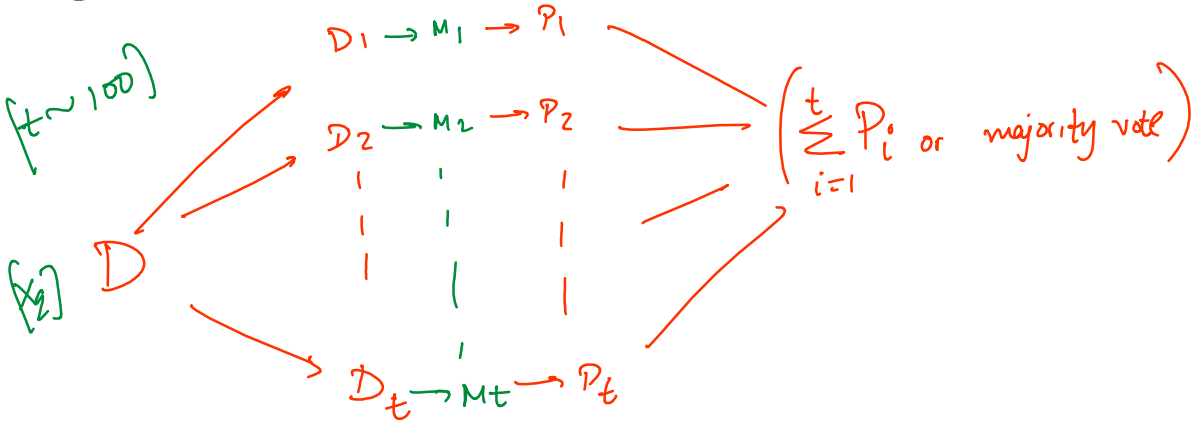
→ same ⇒ (useless work)

Ensemble → same data for all 100 trees → useless (No Benefit)

→ (Diff data) D → [D1 D2 -- D100]

(not feasible) (Data is very expensive)

# From our data (D) → (we make 100 datasets)




Bagging = (Bootstrap + Aggregation)

Bagging is called Random forest when your Model is

Bagging is called Random forest when your Model is D Trees

Bagging = Bootstrap + Aggregation

$(D_1 \; \cdots \; D_{100})$

↓ ↓ ↓

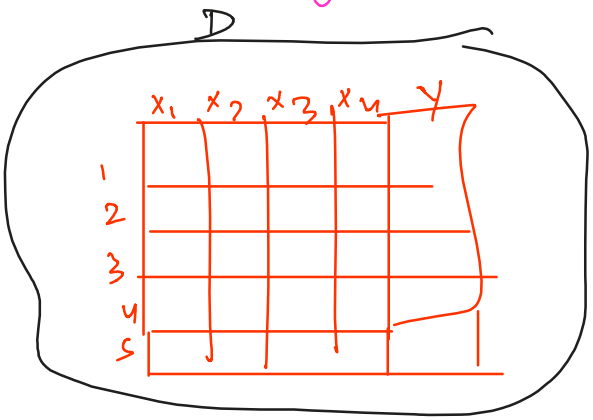
tree trees

1. Decision Tree ⇒ (Bias ↓ (low) or Variance Problem ↓ (high))

Q How to create multiple datasets from 1 data? { Randomness in data }

(I) > (Sampling with replacement) (RS)

(II) > Sample only few columns (CS)




RS+CS

RS+CS

$D_1$ (1) D. Tree

$D_2$ (2) D. Tree

$D_t$

final

Analogy (Class 12th) Aerospace scientist

(I)



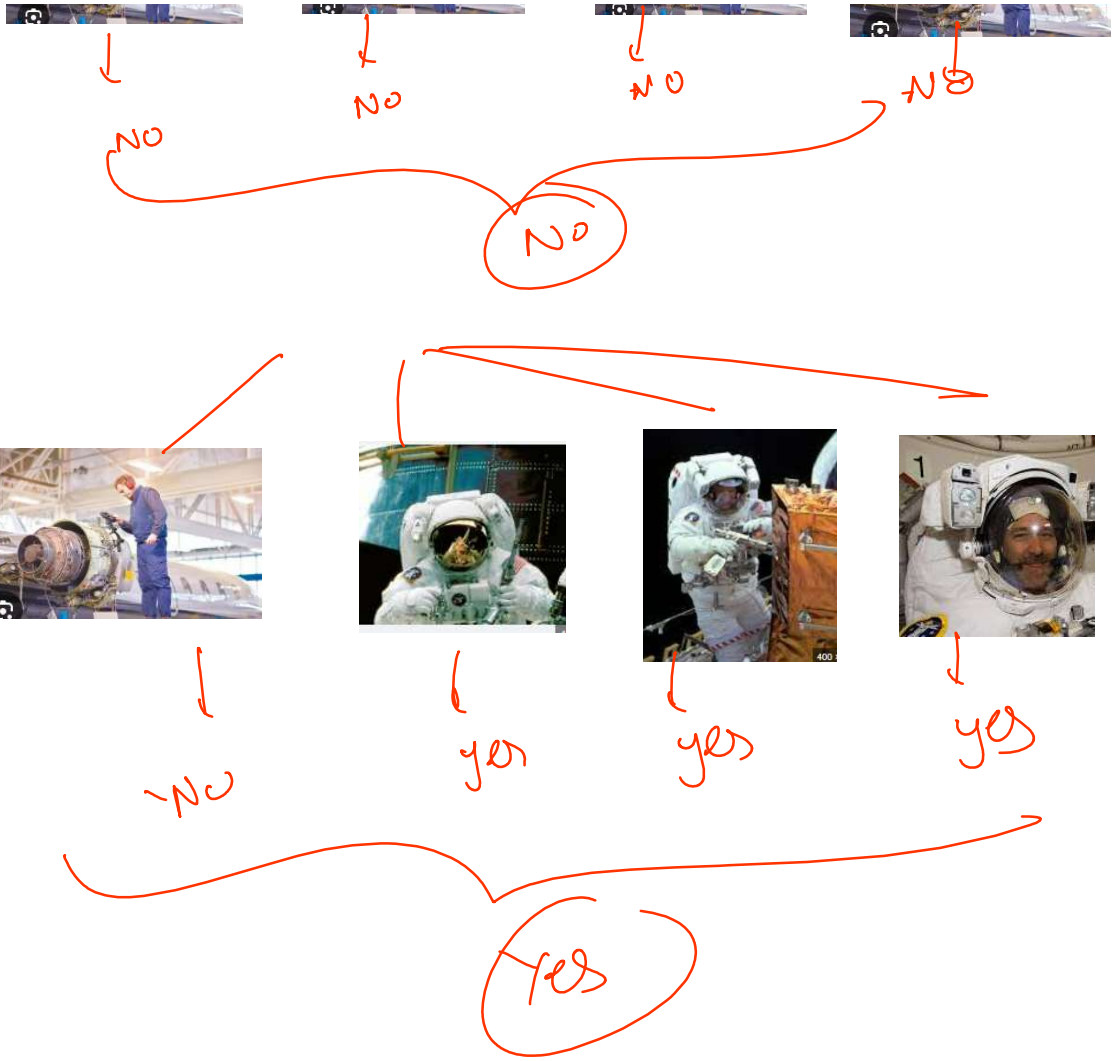






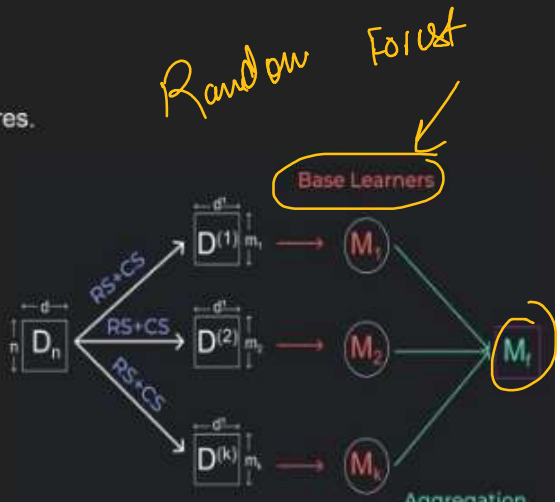
NO

NO



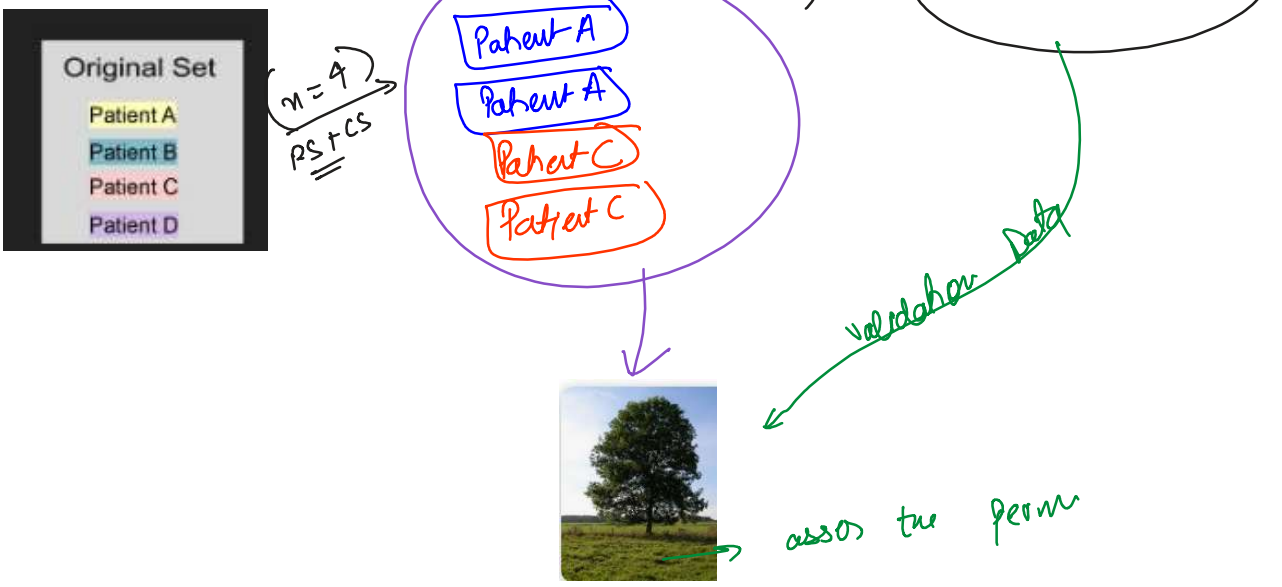

## How to use multiple DTs together?

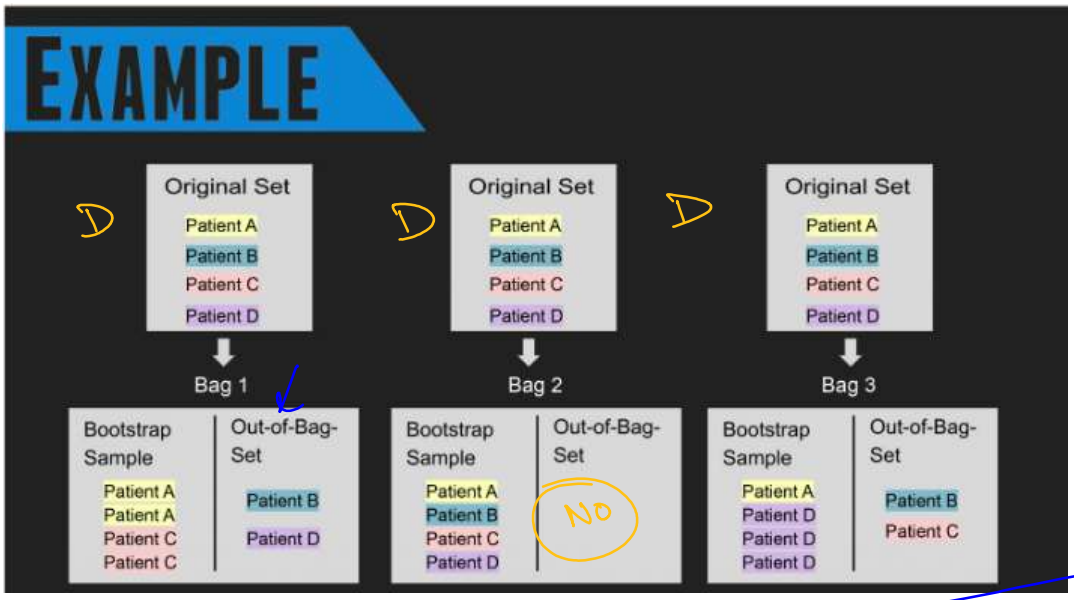
- Assume we have dataset (D) with (n) records and (d) features.
- We sample:
  - RS ($m_1$) data points with replacement
  - CS (d') columns to get dataset ($D_1$)
- Repeat the same (k) times and we get ($D_k$)
- Train (k) different base learners ($M_1$, $M_2$, $M_3$ ..... $M_k$) on these datasets.
- At the end perform Aggregation
  - Classification → Majority Vote
  - Regression → Mean / Median



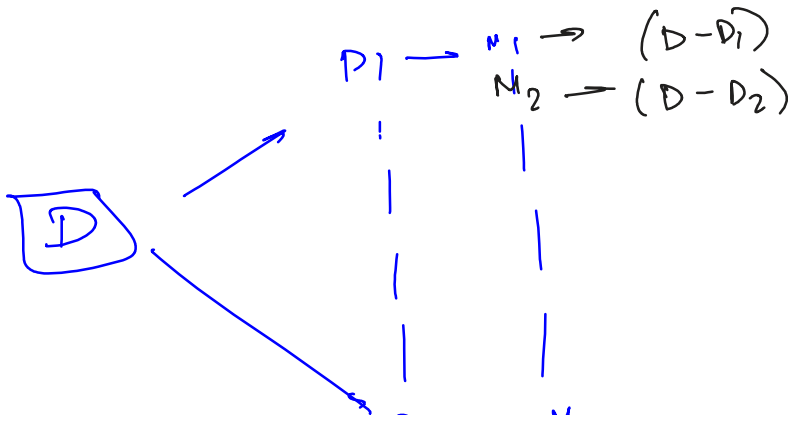

How to validate Random forest ?

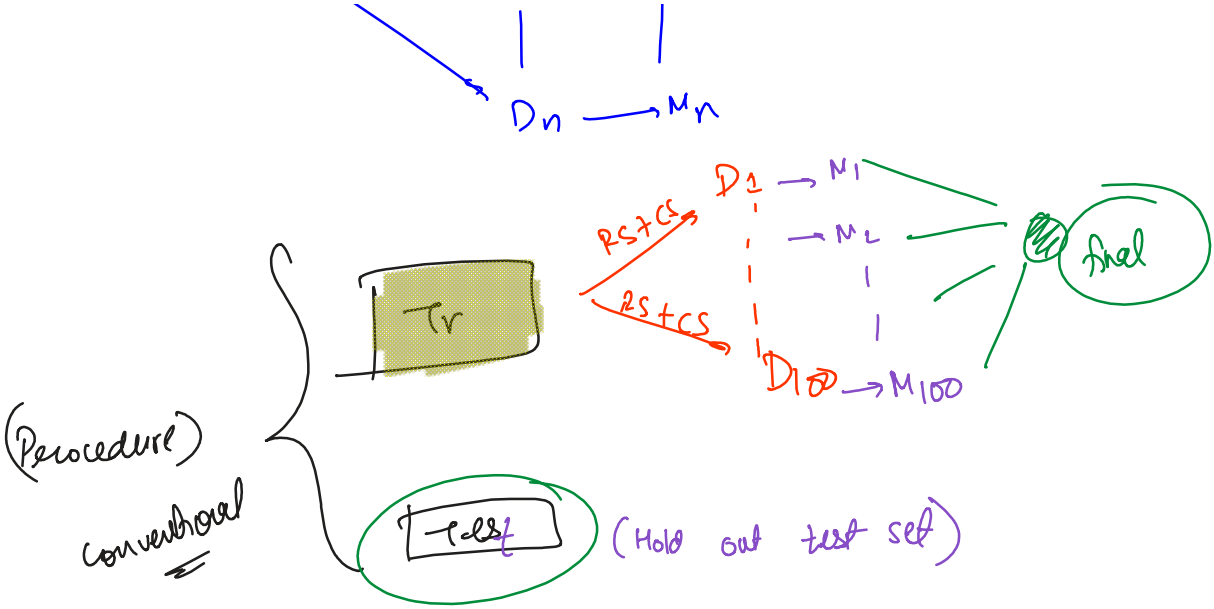
(out of Bag Score)

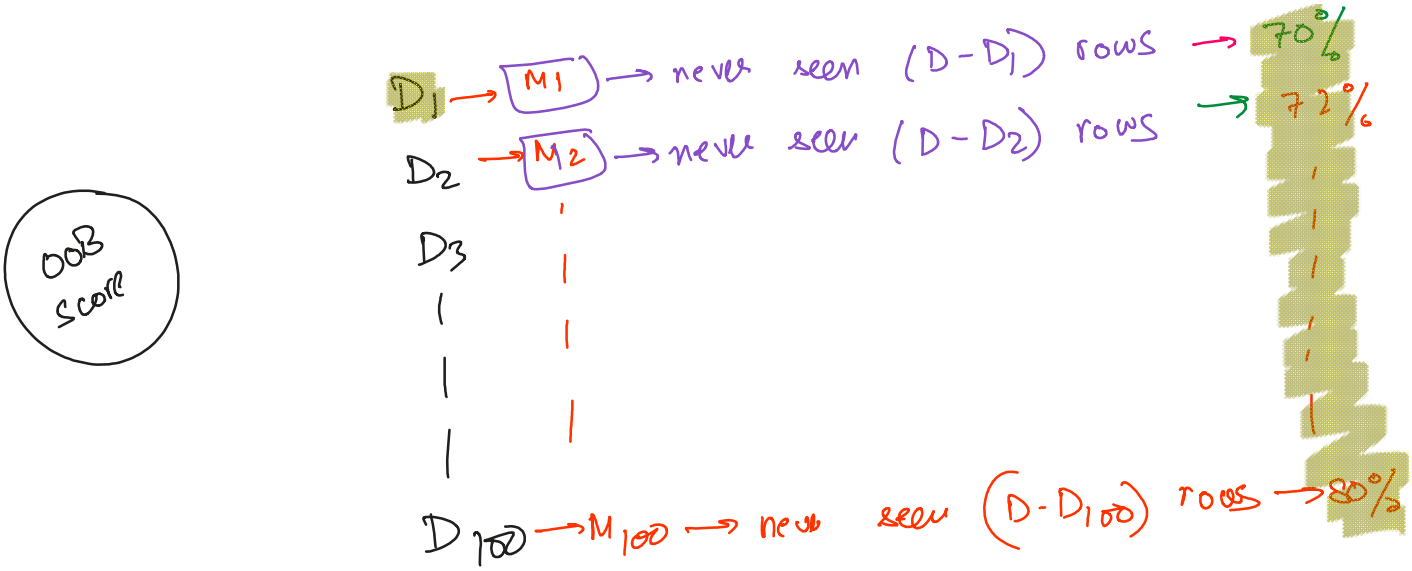



set (D) - set (D1)

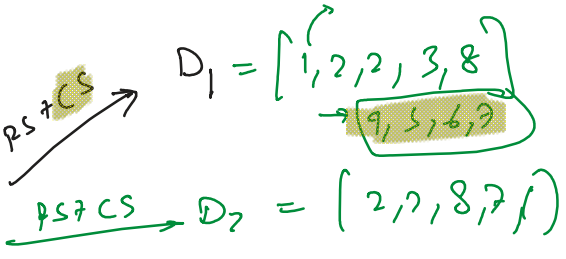





Avg (accuracy) →

| ID | F1 | F2 | Label |
|---|---|---|---|
| 1 | 2.5 | 0.5 | 1 |
| 2 | 1.8 | 1.2 | 1 |
| 3 | -0.5 | 0.3 | 0 |
| 4 | -1.2 | -0.7 | 0 |
| 5 | 0.0 | 2.0 | 1 |
| 6 | 0.2 | -1.5 | 0 |
| 7 | -2.0 | 0.1 | 0 |
| 8 | 1.0 | 1.6 | 1 |



10:17 pm

When you train a Random Forest (RF) you're building many decision trees and combining their votes.
Each tree is trained on its own slightly different view of the data so that the collection of trees is more stable and less likely to make the same mistakes.

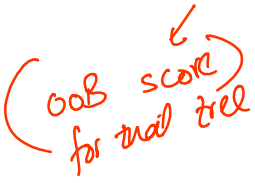
There are two kinds of "unseen data" you should know about: the rows that a particular tree never saw (out-of-bag, or OOB, for that tree), and a completely separate holdout test set that was never used for training or tuning any tree. Both are useful, but they mean different things.

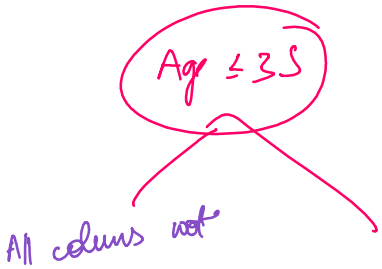
different views




For each tree you pick N rows from your dataset by sampling with replacement. Because you sample with replacement, some rows appear many times in that tree's sample and some rows are not picked at all. The rows not picked for that particular tree are called that tree's *out-of-bag* (OOB) rows

When a tree decides how to split a node, it does not consider all features. It randomly selects a small subset of features and chooses the best split only from that subset. Different splits and different trees will consider different random subsets, so each tree's structure becomes different even if they saw the same rows.

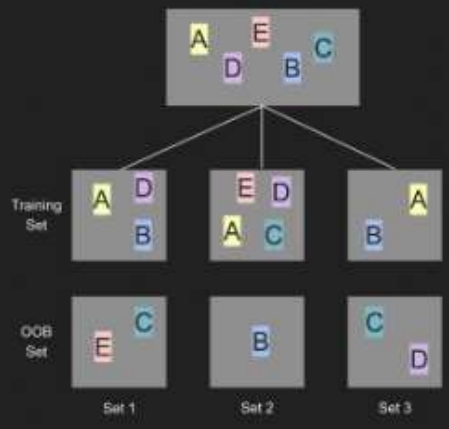


## OOB Score

### How to calculate the OOB score?

Example:-




- Point C will be OOB point for $M_1$ & $M_2$
- After training RF, pass point C through $M_1$ & $M_2$
- Take majority vote / mean of pred
- Compare pred with actual value of label for point C.
- Do this for all points to get OOB Score.

### When should we use OOB Score?

When our dataset is not large enough,

- We can't afford to keep a subset for validation.

So in that case, we estimate overall model performance

- from individual model performances on the OOB points.



(1,2,3,4,5) → 5*

SWR (n=5) → M1

(1,2,2,3,3) 0.63 × 5 ≈ 3 → 62% value unique will be

(1,2,3) 38% ← Tree 1

(2,2,3,4,4) → (2,3,4) ← Tree 2

Logic → How and why does this work ?

Logic → How and why does this work ?

(Bias and Variance)

Bias ↓

Variance ↑

Base Learner (Decision Tree)

(Bagging)

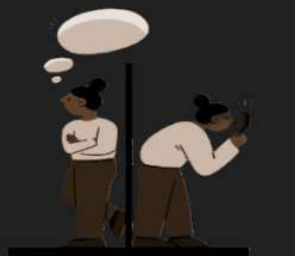
→ Total error ↓

## How to reduce variance?

By using Aggregation.

- Suppose, we have multiple base learners with high variance.
- Predictions vary by ±20% of actual value.
- When we take the average of pred values +ve errors cancel out -ve errors.

*Thus, we are left with smaller residual errors.*



## In Statistical ML,

Error = Bias$^2$ + Variance + Irreducible error

Base Learners ↓ ↑

Bagging ↓ ↓

Due to aggregation, variance decreases without trading of bias.

*Thus, overall error of RF reduces.*

→ Low bias, Low variance

single D. Tree



→ +(20%)




→ +(20%)




+20



−25%



+ 4%

(20 − 25 + 4) = −1%

(Bagging = Bootstrap + Aggregation)

Random Forest (group of trees)

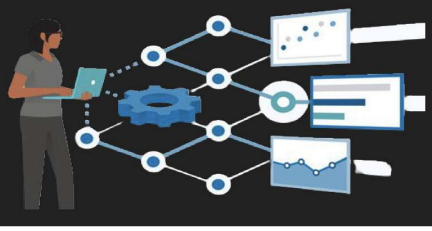
(stochastic RS + CS)

R.F has lot of advantages

## Optimizing our RF Model

The various hyperparameters of random forest are :

1. **n_estimators** - Number of Trees (k)
   *Default = 100*
   (Parallelised) (swf)
2. **max_samples** - Row sample size (m)
   *Default = None, draw all the samples, otherwise m ∈ [0.0 , 1.0]*



3. **max_features** - Number of columns samples (d')
   Can be {"sqrt" , "log2" , None}

100 → √100 → 10

to decide split

3. **max_features** - Number of columns samples (d')
Can be {"sqrt" , "log2" , None}
*Default = "sqrt"*

100 → √100 = 10 to decide split

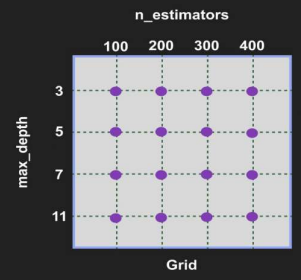



4. **max_depth** - Depth of base learners

## Fine tuning the hyperparameters

**Grid Search**

- Specify a range of hyperparameter values.
- Try every possible combination
- Choose the optimal set




Total Combinations = 4 * 4 = 16

"Brute Force Approach"

**Disadvantages :**

- Computationally expensive
- Time consuming

**If you're performing GridSearchCV on a random forest model for the parameters,**

- **'max_depth' having 3 values and**
- **'min_samples' having 4 values**
- **for 10 cross-validations**
  **How many times model.fit() is called?**

3 × 4 = 12

12 × 10 = 120

4 options

Active Duration (Most preferred: 30 seconds)

Appears for 45 Secs

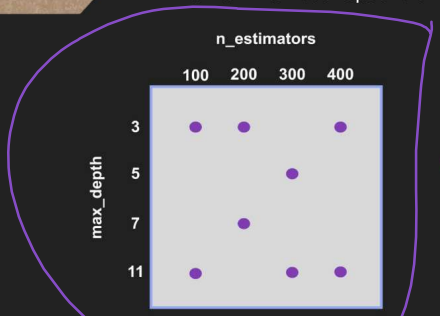
A 1440

B 120

## Fine tuning the hyperparameters (contd.)

**Randomised Search**

- Try random combinations of hyperparameters
- From a finite list of options or from a distribution




Use when dealing with :

1. Large hyperparameter space
2. Limited computational resources

**Advantages :**

- Reduced search space
- Allows faster exploration

max_feature to consider for split

total columns = fixed

D

variance → we used Bagging / RF to overcow this

D. Trees

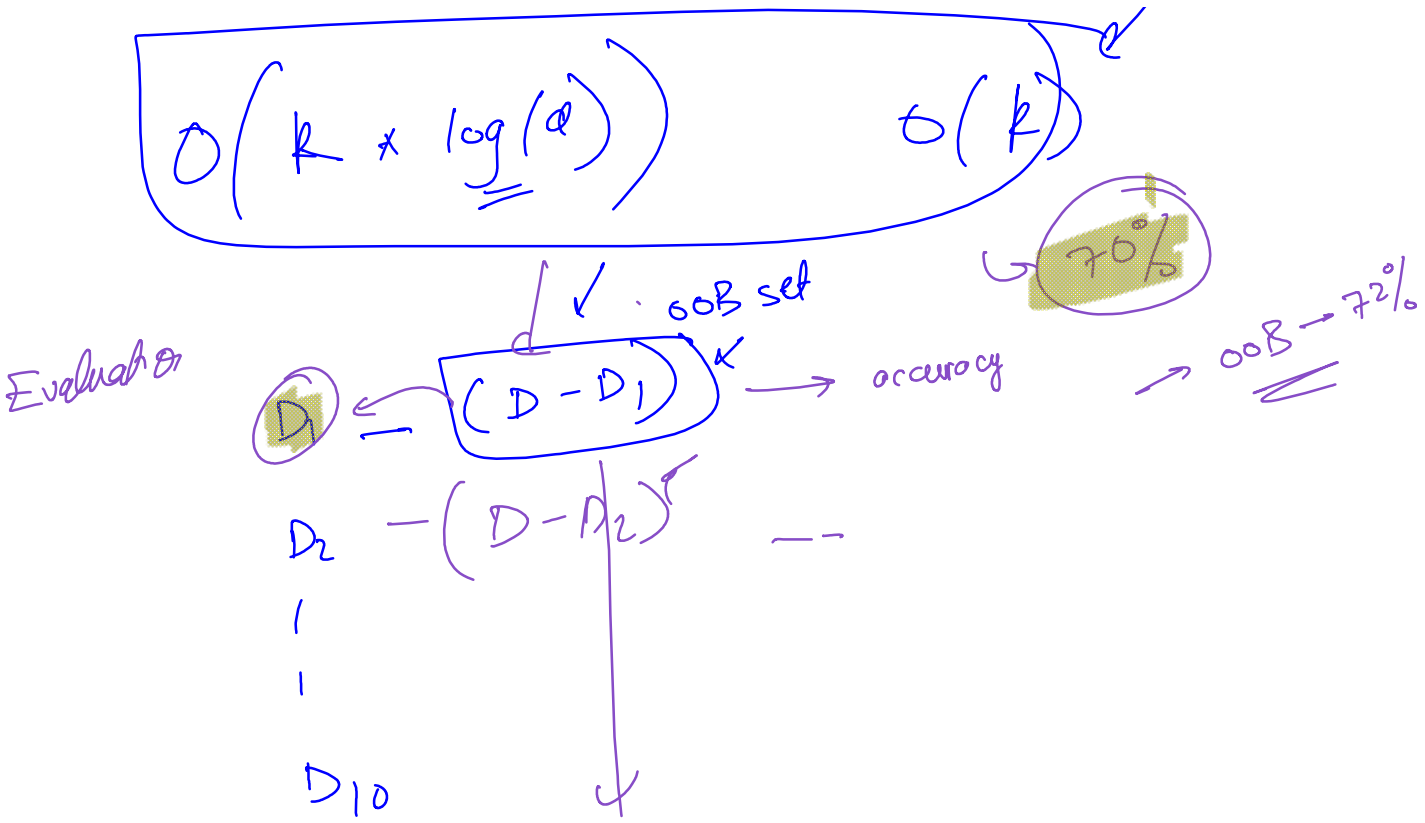
RF → fewer one

fast at inference

* ( n_est = 1000 )

100 trees → inference

O(log(d)) O(100 * log(d)) O(100)


O(k * log(d))
O(k)
70%
OOB set
Evaluator
D1
(D - D1)
accuracy
OOB → 72%
D2
(D - D2)
D10